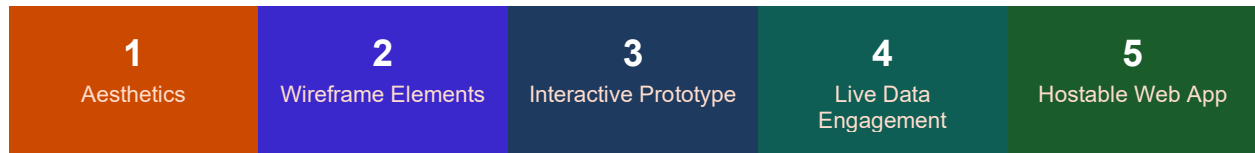


ECON 454 · Spring 2026 · Maxxes

MAXXES

MVP Build Requirements

Wireframe · Aesthetics · Interactive Prototype · Live Data · Hostable Web App



Maxxes is a fan ranking, rewards, and community platform for the live events industry. It connects ticketing history and music streaming behavior into a single verified fan profile — the Superfan Score — and uses that score to unlock tier-based rewards, presale access, and community standing.

MVP Definition

The MVP is a functional, interactive, data-connected web application that can be demonstrated to investors, graded by instructors, and hosted publicly. It must prove five things: the aesthetic is production-quality, every screen is wireframed and built, interactions work, data flows live, and the app runs on a real URL.

What the MVP Is Not

- A static slide deck or PDF mockup.
- A prototype that requires a live presenter to explain every interaction.
- An app that only works locally — it must be deployable to a public URL.
- A full production platform — real Spotify and Ticketmaster API integrations are out of scope. Structured mock data simulates those signals.

02

Aesthetic System — Design Requirements

Colors, typography, spacing, and visual identity that must be implemented

Design Direction — Kyle Rumble

Neutral tone. Clean white surfaces. Gamified but not a video game. Minimalist and easy to navigate. Orange as the single action color. The ranking system must be visually prominent. Every screen should feel like a premium consumer loyalty app — closer to a boarding pass than an arcade.

Color Palette — Official

Color Name	Hex Value	Usage
Action Orange	#FF5C00	Primary CTA buttons, active nav tab, tier highlights, progress fills, badges, all interactive elements
Deep Navy	#2667FF	Score hero backgrounds, featured card overlays, data-heavy surfaces
Deeper Indigo	#3B28CC	Secondary brand moments, Diamond tier, community leaderboard highlights
Light Blue	#ADD7F6	Supporting tints, label text on dark backgrounds
Off-white background	#F2F2F7	Page background, secondary surfaces, inactive chips
Pure white	#FFFFFF	Card backgrounds, pass list rows, primary content surface
True black	#000000	Page titles, primary body text
System gray	#8E8E93	Subtitles, metadata, placeholder text, secondary labels
Border gray	#E5E5EA	Card borders, dividers, row separators
Success green	#1A9E56	Active badge, You're in badge, positive trend indicators
Warning orange-light	#FFF3EE	Badge fills for presale/limited states
Dark card	#111111 → #3D1800	Score hero gradient, featured event cards, image overlays

Typography System

Element	Font	Size	Weight	Usage
Page title	SF Pro / System sans	34px	700 Bold	Maxess, Events, Score, Access, Profile headers
Card title	SF Pro / System sans	22–24px	700 Bold	Score number, featured event title

Section header	SF Pro / System sans	15–17px	700 Bold	Trending now, Upcoming, Challenges
Body / list item	SF Pro / System sans	14–15px	400–500	Pass names, event names, friend names
Metadata / label	SF Pro / System sans	11–13px	400	Dates, venues, subtitles
Micro label	SF Pro / System sans	9–11px	600	Tier badges, chip labels, UPPERCASE tags
Score number	SF Pro / System sans	52–72px	700 Bold	Superfan score display on Home and Score screens
XP number	SF Pro / System sans	20–36px	700 Bold	Streak count, rank, percentile

Spacing & Layout Rules

- Page horizontal padding: 16px on all sides. Never let content touch the screen edge.
- Card border radius: 14–20px. Larger cards (score hero, featured events) use 18–20px. List rows use 0 (inside a parent rounded container).
- Card border: 1px solid #E5E5EA. Active or selected cards use 1.5–2px solid #FF5C00.
- Gap between cards: 8–10px vertical. Gap between sections: 16–20px.
- Bottom navigation height: 49px content + 20px safe area = 69px total. Always sticky.
- Status bar: 44px. Always shown. Always contains time (left) and signal/battery icons (right).
- Touch targets: minimum 44×44px for all tappable elements.

Component Visual Specifications

Component	Visual Spec
Score hero card	Dark gradient background (#111 → #3D1800), white text, flame icon for streak, Elite pill badge (black bg, white text, orange star), footer divider with team and venue
Progress bar	4–5px height, #E5E5EA track, #FF5C00 fill, dot markers at each tier milestone, active dot has orange glow ring
Tier badge pill	Small rounded pill: black background, white text, orange star icon. Used inside score hero and leaderboard rows
Filter chips	Border 1.5px solid #E5E5EA, border-radius 99px. Active chip: border #FF5C00, text #FF5C00
Featured event card	Full-width dark card with gradient overlay, category chip (orange or colored), bookmark icon top-right, title bottom-left, CTA button bottom-right

Pass row	40px icon square (colored bg, matching stroke icon), title + metadata left, status badge right, chevron far right
Status badge	Pill shape, color-coded: green for Active, orange for Starts soon/Limited, gray for Waiting list
Challenge card	Two-part card: header with thumbnail image + title + progress bar; footer with checkbox task list
Leaderboard row	Rank number (orange if you), avatar circle, name (orange + bold if you), score, trend arrow with delta
Bottom navigation	5 tabs: Home, Events, Score, Access, Profile. Active tab icon and label in #FF5C00. All others #8E8E93

The Maxxes MVP consists of five screens connected by a persistent bottom navigation bar. Each screen has a defined set of components, states, and interactions. All five must be fully implemented — not mocked — for the MVP to be complete.

Home

- Status bar — time left, icons right
- Page header — 'Maxess' title, 'Your fan profile' subtitle, notification bell with orange dot
- Music / Sports Events toggle — two-state pill toggle, orange fill on active side
- Superfan Score hero card — dark gradient, score (91/100), streak (flame + number + label), Elite badge, Top % stat, team + venue footer
- Progress bar card — bordered card, 'Progress to Diamond+' label, XP to next tier (right-aligned orange), 5-dot milestone track (Bronze/Silver/Gold/Platinum/Diamond), tier labels + XP values below
- Upcoming section header — 'UPCOMING' label + 'View all' orange link
- 2-column upcoming event grid — each card: colored image area, category chip (GAME ACCESS / VENUE ACCESS), bookmark icon, event title, date footer with orange arrow button
- Climb the Ranks card — 3 overlapping avatars, headline + subtitle, 'See leaderboard' pill button
- Bottom navigation — 5 tabs, Home active

Events

- Status bar
- Page header — 'Events' title, 'Discover music + sports for you' subtitle, notification bell
- Search bar — rounded gray input, magnifier icon, filter button (sliders icon) right
- Filter chip row — scrollable: For you (active orange), Music, Sports, This week
- Featured hero card — full width, dark overlay, FEATURED chip, large event title, subtitle, date + location metadata row, Notify me orange button
- Trending now section — 2-column grid, each card with MUSIC or SPORTS chip, bookmark, title, date/venue metadata
- Upcoming for you section — list card with thumbnail squares, event name, metadata, status badge (Starts in 18h / You're in), chevron
- Near you this week — horizontal scrollable date strip (TODAY highlighted orange), 2-column event grid below
- Bottom navigation — Events active



Score

- Status bar
- Page header — 'Score' title, 'Community ranking + reward challenges' subtitle, notification bell
- Score summary card — bordered card with 4 sections: large score (91/100) + Elite badge | This week delta (+12, green) | Your rank (#18 in LA) | Percentile (Top 3%)
- Community leaderboard card — title + 'LA / This week' dropdown filter, 5 leaderboard rows (rank, avatar circle, name, score, trend arrow + delta), your row highlighted orange background, 'View full leaderboard' orange link at bottom
- Friends activity section — horizontal scroll of friend cards (avatar, name, activity description, pts/rank change in orange)
- Challenges unlocking rewards section — challenge cards with: thumbnail image, Unlock reward chip, title, description, X/Y tasks complete label, progress bar; task checklist with filled/empty checkboxes; Limited badge top-right
- 'View all challenges' orange link
- Bottom navigation — Score active



Access

- Status bar
- Page header — 'Access' title, 'Presales, perks, and member passes' subtitle, notification bell
- Elite/Diamond Access strip — bordered card with 3 KPIs: Active count | Expiring soon count | Top tier label; each with orange icon circle
- Featured access section — two feature cards: each with dark image background, TOP PICK or LIMITED chip, title, subtitle, time indicator + CTA button (View pass / Claim access)
- Your passes section — 'Your passes' title + 'View all' link; list card with 3 rows: icon square, pass name, metadata, status badge (Active green / Starts soon orange / Waiting list gray), chevron
- Perks by tier section — 3-column tier grid: General (gray icon), Superfan (purple icon), Elite/Diamond (orange icon, orange border active); each column has tier name, perk description, colored underline bar
- Unlock next card — bordered card with outlined orange icon, title, subtitle, X/Y progress label (orange), progress bar, chevron
- Bottom navigation — Access active



Profile

- Status bar
- Page header — 'Profile' title, settings gear icon right
- Profile header — large avatar circle (initials), fan name, member since date, current tier badge
- Stats row — 3 metric cards: Total XP | Events attended | Streak days
- Tier progress card — same progress bar as Home screen, current tier highlighted

-
- My artists section — scrollable row of artist chips with listen percentile badges
 - My venues section — scrollable row of venue cards with attendance count
 - Recent activity list — chronological feed of XP-earning events with source icon, description, XP amount, timestamp
 - Share profile button — full-width orange button
 - Bottom navigation — Profile active

04

Interactive Prototype Requirements

What must respond, what must update, what must feel real

An interactive prototype is not a collection of static images with hotspots. Every interaction listed below must be implemented in code. When a user taps a button, state must change. When data updates, the UI must reflect it. The prototype must be indistinguishable from a real app during a 5-minute demo.

Required Interactions by Screen

Screen	Interaction	Expected Behavior
Home	Music / Sports Events toggle tap	Switches score card content between music stats and sports stats. Score, streak, team/artist, and upcoming events all update.
Home	Upcoming event card tap	Navigates to event detail view or expands to show more info. Date, venue, access type visible.
Home	See leaderboard tap	Navigates to Score screen, leaderboard scrolled into view.
Home	Notification bell tap	Shows notification panel or badge count clears.
Events	Filter chip tap	Filters event list. Only events matching the selected category display. Active chip turns orange.
Events	Search input tap + type	Filters event list in real time. No results state must exist.
Events	Date strip chip tap	Filters Near You events to selected date. Selected chip turns orange.
Events	Notify me button tap	Button state changes to 'Notified ✓' with green confirmation. Cannot be tapped again.
Events	Bookmark icon tap	Icon fills to indicate saved. Tap again to unsave. State persists during session.
Events	Event card tap	Expands or navigates to event detail with full info.
Score	Leaderboard filter dropdown	Changes filter between LA / This week, LA / This month, All cities. List reorders.
Score	View full leaderboard tap	Expands leaderboard to show more rows.
Score	Challenge task checkbox tap	Marks task complete. Progress bar and X/Y label update. If all tasks complete, card shows 'Reward unlocked' state.
Score	Friend card tap	Shows friend's score profile or mini card.
Access	Featured card CTA tap	View pass → shows pass detail. Claim access → shows confirmation or waiting state.

Access	Pass row tap	Expands or navigates to pass detail showing full info, QR code placeholder, status.
Access	Tier column tap	Highlights selected tier, shows perk detail.
Access	Unlock next chevron tap	Shows what is needed to unlock the next reward.
Navigation	Bottom nav tab tap	Navigates between all 5 screens. Active tab turns orange. Transition animates.
Navigation	Back gesture / button	Returns to previous screen where applicable.

Required State Management

- **Fan profile state:** name, XP total, current tier (Bronze/Silver/Gold/Platinum/Diamond), streak days, rank, percentile — must be accessible from any screen.
- **Toggle state:** Music vs. Sports Events selection persists while user is on Home. Resets on app reload unless stored.
- **Filter state:** Active filter chip, selected date, search query — all update the displayed list in real time.
- **Challenge completion state:** Checked tasks, progress bar percentage, reward unlock status — must persist during session.
- **Notification state:** Notify me button, bookmark icons — toggled state must persist during session.
- **Leaderboard state:** Selected filter (city/time period) determines which data slice renders.

Animation & Transition Requirements

- Screen transitions: 300ms ease slide or fade. Not instant jumps.
- Toggle switch: 200ms ease slide of the orange pill between Music and Sports.
- Progress bar: animates from 0 to actual value on screen mount (600ms ease-out).
- Score number: counts up from 0 to actual value on Home screen load (800ms).
- Challenge progress bar: animates to fill percentage when screen loads.
- Status badge: no animation — renders immediately at correct state.
- Bottom nav active indicator: 150ms fade/color transition on tab change.

05

Live Data Architecture — Passive Data Engagement

How the prototype connects to and displays real data without full API integration

Live data engagement means the prototype reads from a real data source at runtime — not hardcoded values. For the MVP, this means a structured JSON data layer hosted alongside the app (or on a lightweight backend) that the UI fetches on load. This is passive live data: the data can be updated by the team and the app will reflect those changes the next time a user loads it.

Live Data Strategy

The MVP uses seed JSON files as the data source. These files are real structured data — not embedded in the code. The app fetches them at runtime. This means the team can update fan scores, events, leaderboard positions, and challenges without touching the UI code. It also means the data layer is ready to be swapped for a real API when integrations are added.

Data Files Required

File	Contents	Consumed By
fan-profile.json	fan_id, name, xp_total, current_tier, streak_days, rank, percentile, top_artist, top_venue, events_attended, xp_breakdown by signal type	Home score card, Progress bar, Profile screen, Score summary
events.json	event_id, title, subtitle, category (music/sports), date, time, venue, city, access_type, status, featured flag, image_color	Events screen — all sections, Home upcoming cards
leaderboard.json	Array of fans: rank, name, score, delta, is_me flag, city, time_period	Score screen leaderboard, Score filter dropdown
challenges.json	challenge_id, title, subtitle, reward_name, tasks (array with is_complete flag), progress_fraction, is_limited	Score screen challenges section
passes.json	pass_id, name, venue, date, status (active/starts_soon/waiting), pass_type, xp_tier_required	Access screen passes list
friends.json	Array: name, score, activity_label, delta, delta_type (up/down/neutral)	Score screen friends activity strip
tiers.json	tier definitions: name, xp_threshold, perks list, color hex	Progress bar, Access perks grid, Profile tier card — single source of truth for all tier logic

Data Fetching Architecture

Layer	Implementation	Why
Data files	JSON files in /public/data/ folder of the SvelteKit project	Served as static assets — no server required. Fetchable from any browser.
Fetch on load	Each screen fetches its required JSON files in the onMount() lifecycle hook	Data loads fresh every visit. Team can update JSON files and users see new data.
Loading state	Skeleton placeholder shown while fetch is in progress	Prevents layout shift. Looks professional during demo.
Error state	Graceful fallback message if fetch fails	Demo cannot crash mid-presentation.
Fan profile store	Svelte writable store holds fan-profile.json data after first fetch	Shared across all screens without re-fetching. Score, tier, XP accessible everywhere.
Tier logic	Derived from tiers.json — not hardcoded	Changing XP thresholds in one file updates everywhere. Single source of truth.
Future swap path	Replace fetch('/data/file.json') with fetch('https://api.maxxes.com/endpoint')	No UI code changes needed when real API is ready. Clean separation of data and presentation.

XP Scoring — Client-Side Logic

The XP scoring calculation runs in the browser using data from fan-profile.json. This makes the score transparent, testable, and auditable during a demo.

```
// src/lib/domain/xp.ts
export const TIERS = [
  { name: 'Bronze', min: 0, max: 2499 },
  { name: 'Silver', min: 2500, max: 4999 },
  { name: 'Gold', min: 5000, max: 9999 },
  { name: 'Platinum', min: 10000, max: 24999 },
  { name: 'Diamond', min: 25000, max: Infinity },
];

export function classifyTier(xp: number): string {
  return TIERS.find(t => xp >= t.min && xp <= t.max)?.name ?? 'Bronze';
}

export function xpToNextTier(xp: number): number {
  const next = TIERS.find(t => t.min > xp);
  return next ? next.min - xp : 0;
}

export function tierProgress(xp: number): number { // 0.0 - 1.0
  const tier = TIERS.find(t => xp >= t.min && xp <= t.max);
  if (!tier || tier.max === Infinity) return 1;
  return (xp - tier.min) / (tier.max - tier.min);
}
```


The prototype must be deployable to a public URL by the end of the capstone. This means it runs in any browser without installation, is accessible on desktop and mobile, and remains live after the presentation. The stack choices below are specifically selected for zero-cost hosting, minimal configuration, and a deployment path that a student team can execute in under 30 minutes.

Recommended Technology Stack

Layer	Technology	Why This Choice
App framework	SvelteKit + TypeScript	Generates a static build that deploys to any host. Lightweight, fast, no server required for MVP.
Styling	Tailwind CSS	Utility classes match the pixel-perfect spacing and color system. No custom CSS framework to maintain.
UI components	shadcn-svelte	Pre-built accessible components (buttons, badges, progress, drawers). Matches Maxxes aesthetic with minimal override.
Icons	Lucide Svelte	Clean outline icons for all nav, action, and status uses. Matches Kyle's design direction.
Charts	layerchart	Progress bars, leaderboard bars, XP breakdown — lightweight and composable.
Data	Static JSON in /public/data/	No database required. Fetched at runtime. Updatable by anyone with repo access.
Validation	Zod	Validates JSON data shape on fetch. Prevents runtime crashes from malformed data.
Date/time	date-fns	Countdown to event, relative time labels (Opens in 18h), date formatting.
Hosting	Vercel or Netlify	Free tier. Connects to GitHub. Every push auto-deploys. Public URL generated instantly.
Build output	@sveltejs/adaptor-static	Generates pure HTML/CSS/JS. No server. Works on any static host including Vercel, Netlify, GitHub Pages.
Testing	Vitest + Playwright	Unit tests for XP scoring and tier logic. E2E tests for the demo flow path.

Project Structure

```

maxxes-app/
├── src/
│   ├── routes/
│   │   ├── +layout.svelte      ← bottom nav + shared shell
│   │   ├── +page.svelte       ← Home screen
│   │   ├── events/+page.svelte ← Events screen
│   │   ├── score/+page.svelte ← Score screen
│   │   ├── access/+page.svelte ← Access screen
│   │   └── profile/+page.svelte ← Profile screen
│   ├── lib/
│   │   ├── components/        ← all reusable UI components
│   │   │   ├── ScoreHero.svelte
│   │   │   ├── ProgressBar.svelte
│   │   │   ├── EventCard.svelte
│   │   │   ├── LeaderboardRow.svelte
│   │   │   ├── ChallengeCard.svelte
│   │   │   ├── PassRow.svelte
│   │   │   └── BottomNav.svelte
│   │   ├── domain/
│   │   │   ├── xp.ts          ← tier classify, progress, next threshold
│   │   │   └── types.ts       ← Fan, Event, Challenge, Pass interfaces
│   │   ├── stores/
│   │   │   └── fan.ts         ← writable Svelte store for fan profile
│   │   └── styles/
│   │       └── tokens.css      ← color variables, spacing, typography
│   └── static/
│       └── data/              ← all JSON data files (live data layer)
│           ├── fan-profile.json
│           ├── events.json
│           ├── leaderboard.json
│           ├── challenges.json
│           ├── passes.json
│           ├── friends.json
│           └── tiers.json
├── tests/
│   ├── unit/xp.test.ts
│   └── e2e/demo-flow.spec.ts
├── svelte.config.js          ← adapter-static configuration
└── package.json

```

Deployment Steps — Zero to Live URL

1. Create the SvelteKit project: `npx sv create maxxes-app`, select TypeScript + Tailwind.
2. Add adapter-static: `npm install @sveltejs/adapter-static`, configure in `svelte.config.js`.
3. Add shadcn-svelte: `npx shadcn-svelte@latest init`.
4. Add remaining packages: `npm install layerchart zod date-fns @lucide/svelte`.
5. Place all JSON data files in `/static/data/`.
6. Build all five screen routes and components.
7. Run `npm run build` — generates `/build` folder of pure static files.
8. Push to GitHub repository (any name).

9. Connect GitHub repo to Vercel (vercel.com) — sign in with GitHub, click Import, select repo.
10. Vercel auto-detects SvelteKit + adapter-static. Click Deploy.
11. Public URL is live within 60 seconds: <https://maxxes-app.vercel.app> (or custom domain).
12. Every future git push auto-redeploys. Data updates via JSON file edits in the repo.

Custom Domain (Optional)

If the team wants maxxes.app or a similar URL, Vercel's free tier supports custom domains. Purchase the domain (approximately \$10–15/year), add it in Vercel's domain settings, and update DNS records at the registrar. The app is then live at a permanent, professional URL.

07

MVP Acceptance Criteria

The complete checklist — every item must pass for the MVP to be complete

The following checklist is the standard against which the Maxxes MVP will be evaluated. Every item must pass. Items marked with a role name indicate the primary owner of that requirement.

Aesthetic ✓

Criterion	Owner	Pass Condition
Orange (#FF5C00) is the sole action color	Kyle	Every button, active tab, progress fill, and CTA uses orange exclusively
White card surfaces with gray borders	Kyle / Luke	No colored card backgrounds except hero cards and featured events
Score hero card is dark with correct gradient	Luke	Matches Kyle's mockup: dark brown-black gradient, white text
Typography matches spec	Luke	34px bold page titles, 52–72px score numbers, 8–11px micro labels
16px page padding on all screens	Luke	No content touches screen edge
All touch targets minimum 44×44px	Luke	Verified on mobile viewport
Tier badge renders in black pill with orange star	Kyle / Luke	Matches mockup exactly on all screens where it appears

Wireframe Completeness ✓

Criterion	Owner	Pass Condition
All 5 screens built and navigable	Luke / Kayah	Home, Events, Score, Access, Profile all render without error
Bottom navigation present on all screens	Luke	5 tabs, correct active state, correct icons
Score hero card on Home screen	Luke / Ethan	Score, streak, badge, team, venue all display from data
Progress bar with Bronze–Diamond tiers	Luke / Ethan	5 milestones labeled correctly with XP values, fill matches current XP
Music / Sports toggle on Home	Luke	Both states render different content
2-column upcoming event grid on Home	Luke / Ethan	At minimum 2 cards with chip, title, date, arrow button
Search + filter chips on Events	Luke	Search filters list, chips filter by category

Featured hero card on Events	Luke / Ethan	Full-width, dark overlay, correct chip, Notify me button
Leaderboard with 5 rows on Score	Luke / Ethan	Your row highlighted orange, rank, name, score, delta all display
Challenge cards with task checklists on Score	Luke / Ethan	Minimum 2 challenges, each with tasks and progress bar
Featured access cards on Access	Luke / Ethan	Minimum 2 featured cards with correct chip and CTA
Pass list on Access	Luke / Ethan	Minimum 3 passes with correct status badges
Tier perks grid on Access	Luke / Kyle	3 columns, active tier highlighted orange border
Profile screen with all sections	Luke / Ethan	Avatar, stats row, tier progress, activity feed all present

Interactivity ✓

Criterion	Owner	Pass Condition
All bottom nav tabs navigate correctly	Luke	Tapping each tab renders the correct screen
Toggle switches between Music and Sports content	Luke / Ethan	Content changes — not just label
Filter chips filter event list in real time	Luke	Tapping Music shows only music events
Notify me button changes state on tap	Luke	Button becomes 'Notified ✓' and cannot be tapped again
Bookmark icons toggle on tap	Luke	Fill state changes; persists during session
Challenge checkboxes update progress bar	Luke / Ethan	Checking a task increments the X/Y label and bar width
Score number animates on load	Luke	Counts up from 0 to value over ~800ms
Progress bar animates on load	Luke	Fills from left to actual percentage over ~600ms
Screen transitions animate	Luke	Not instant jumps — slide or fade over ~300ms
Leaderboard filter changes data	Luke / Ethan	Dropdown selection reorders/filters the list

Live Data ✓

Criterion	Owner	Pass Condition
All data fetched from JSON files at runtime	Ethan / Luke	No hardcoded values in component code — all values come from fetched data
Loading skeleton shown during fetch	Luke	Skeleton placeholder renders before data arrives

No crash if fetch fails	Ethan / Justin	Error state renders gracefully
Fan profile store shared across all screens	Ethan	Score, tier, XP consistent on Home, Score, Profile without re-fetch
Tier classification uses tiers.json	Ethan	Changing XP threshold in JSON updates tier across all screens
XP scoring function is separate from UI	Ethan	classifyTier(), xpToNextTier(), tierProgress() in domain/xp.ts
JSON data is realistic and populated	Ethan	25+ fan profiles in leaderboard, 10+ events, 3+ challenges, 5+ passes
Data can be updated without touching UI code	Ethan	Edit JSON file → push to repo → app reflects change on next load

Hostable Web App ✓

Criterion	Owner	Pass Condition
App builds without error (npm run build)	Kayah / Luke	Zero build errors, /build folder generated
Static adapter configured correctly	Kayah	svelte.config.js uses adapter-static
App is live on a public URL	Kayah	URL accessible from any device without login or installation
App works on mobile viewport (375px)	Luke	All 5 screens render correctly at iPhone screen width
App works on desktop viewport (1280px+)	Luke	App centers or scales appropriately — not broken at wide width
Page load time under 3 seconds	Kayah / Luke	Tested on standard connection. JSON files are small.
Data updates reflect on next load	Ethan / Kayah	Edit JSON, push to GitHub, Vercel redeploys, new data visible
URL is shareable and persistent	Kayah	Link works for anyone, anytime — not just during demo session

08

Role Ownership Matrix

Who builds what — no overlap, no gaps

Role	Primary Ownership	Key Deliverables for MVP
Kayah Whipps — Technical Lead	Architecture, deployment, data contracts, scope control	svelte.config.js with adapter-static, GitHub repo setup, Vercel deployment, data contract document defining all JSON schemas, scope decision log
Kyle Rumble — Design / Creative	Visual identity, component specs, design QA	Figma wireframes or annotated mockups for all 5 screens, style guide (colors, type, spacing), design QA checklist verifying every screen matches spec
Luke Pollard — Front End	All Svelte components, all 5 screens, all interactions, animations	Every component file, all 5 screen routes, all interactions implemented, mobile viewport verified, score animation, progress bar animation
Ethan King — Back End / Data	All JSON data files, XP domain logic, Svelte stores, data fetching	All 7 JSON files with realistic seed data, domain/xp.ts, stores/fan.ts, fetch logic in each screen, loading and error states
Justin Hurst — QA / Documentation	Testing, evidence trail, QA checklist execution, final documentation	Unit tests for XP logic, E2E demo flow test, completed acceptance checklist with pass/fail evidence, contribution summary, final presentation support

The Standard

A completed Maxxes MVP is a live URL that anyone can open on their phone, navigate through five screens, see their fan score update from real data, tap through challenges and passes, and understand the product without a single word of explanation from the team. That is the bar.

Everything in this document exists to meet it.